

PEC 2: Juegos

Presentación

Segunda PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajaran las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

Objetivos

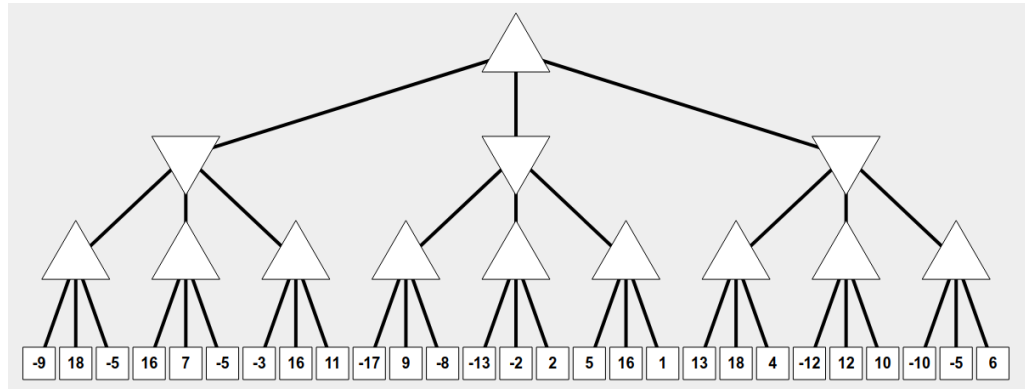
Esta PEC pretende evaluar vuestros conocimientos sobre búsquedas, su formalización y estrategias relacionadas.

Descripción de la PEC/práctica a realizar

Esta PAC tendrá como objeto los algoritmos de minimax y la poda alfa-beta, y su codificación en Common Lisp.

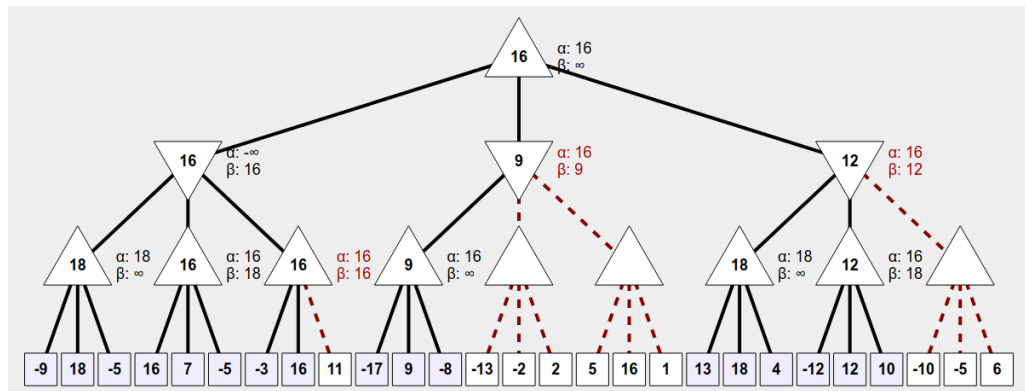
Pregunta 1 (30%):

Suponiendo que el nodo raíz es un nodo max, aplicar manualmente el algoritmo de la poda alfa-beta sobre este árbol:



Determinar qué rama elegirá el jugador, y qué ramas serán podadas.

Solución:



El jugador escogerá la rama izquierda, y las ramas podadas aparecen en rojo.

Pregunta 2 (30%):

Ahora utilizaremos los programas en Common Lisp **minimax** y **minimax-alpha-beta** que pueden encontrarse en el archivo adjunto. Tanto el algoritmo minimax como la poda alfa-beta recorren los nodos del árbol de izquierda a derecha. Modificar los programas dados para que este recorrido lo hagan de derecha a izquierda. Utilizarlos con el árbol de

la pregunta 1 y comparar los resultados con los obtenidos, manualmente, en la pregunta 1. ¿Ha habido algún cambio en el resultado del minimax? ¿Y de la poda alpha-beta? Justificar las respuestas y razonar de manera genérica, independientemente del árbol analizado.

Solución:

Bastará con sustituir `(hijos árbol)` o `(hijos nodo)` en los correspondientes programas con `(reverse (hijos árbol))` o `(reverse (hijos nodo))`.

Así, el resultado por el algoritmo minimax será idéntico, ya que la jugada elegida no cambia cuando se hace el recorrido de izquierda a derecha o de derecha a izquierda. En cambio, en la poda alfa-beta sí que encontramos diferente resultado. Después de hacer funcionar el programa adjunto `minimax-alpha-beta` en el árbol de la pregunta 1 tenemos:

=> -9 18 -5 16 7 -5 -3 16 -17 9 -8 13 18 4 -12 12 10

y si hacemos el recorrido de derecha a izquierda obtenemos:

6 -5 -10 10 4 18 1 16 5 2 -2 -13 11 16 -3 -5 7 16 -5 18

que invertido, para poder comparar, es:

=> 18 -5 16 7 -5 -3 16 11 -13 -2 2 5 16 1 18 4 10 -10 -5 6

Observemos que se han podado menos nodos.

No habrá ningún cambio en el algoritmo del minimax ya que va haciendo un cálculo donde los argumentos dependen del nivel completo del árbol, así que no importará el orden de exploración del árbol. En cambio, en la poda los cálculos de alfa y beta sí dependen del orden en que se explora el árbol, ya que van cogiendo los valores de las hojas pertinentes para establecer las cotas alfa y beta correspondientes.

Pregunta 3 (40%):

En un artículo de 1983 de la revista Artificial Intelligence (*A comparison of minimax tree search algorithms*, Murray S. Campbell, T.A. Marsland, Artificial Intelligence, Vol.20, n.4, 1983, pp. 347-367) aparece una propuesta de poda alfa-beta (en la figura 2 del artículo). Esta propuesta es, traducida a Common Lisp:

```
(defun mystery (p x1 x2)
  (if (hoja p)
      (let ((val (evalua p)))
        (format t "~A " val)
        val)
      (let ((w (hijos p))
            (m x1))
        (dolist (q w m) (let ((k (- (mystery q (- x2) (- m)))))
                          (if (> k m) (setf m k)
                              (if (>= m x2) (return m)))))))))
```

pero no funciona bien en el caso general (de hecho, no funciona la poda, pero tampoco el valor minimax). Investigar para qué tipo de árboles de juegos este programa no funciona (haga distintas pruebas con diferentes árboles). Se tiene el algoritmo correcto ya implementado, así que encontrar las soluciones correctas no cuesta nada. Comparar con las soluciones que genera este programa y encontrar el patrón que indica cuando no funciona correctamente. Hay que tener en cuenta que, en general, la raíz puede ser min o max, y el árbol puede tener un número arbitrario de niveles. En el archivo adjunto se encuentran varios árboles codificados para poder hacer pruebas.

Solución:

Este algoritmo sólo funciona bien con árboles completos con max en la raíz y mismo número de tiradas de los dos jugadores, es decir, los subárboles de altura 1 deben ser árboles min. Si los árboles tienen diferentes niveles no funcionará bien en general.

Recursos

Para realizar esta PEC el material imprescindible son los temas 2, 3, 4 y 6 del Módulo 2.

Criterios de valoración

Problema 1: 30%

Problema 2: 30%

Problema 3: 40%

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC2 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **9 de Noviembre** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.